Warehouse Receiving & Inventory Reconciliation Engine — Project Build Note

Oct 5, 2026

1. Purpose and scope

The engine automatically reconciles what a business ordered, what the supplier delivered, what the warehouse physically counted and what the supplier invoiced, then acts on the result. It was built in n8n (self-hosted, version 2.30.7) as the Group 11 capstone for the TechCrush AI Automation programme, Cohort 8.

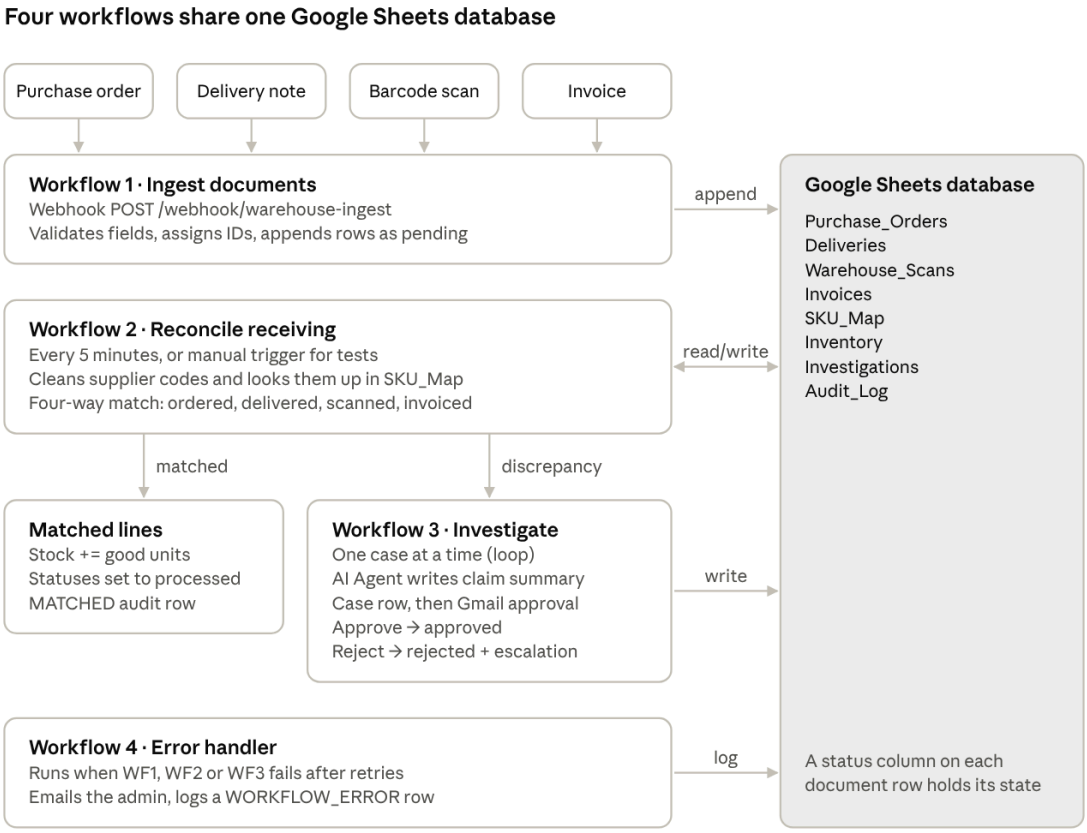
The brief asks for five things, all of which the system now does:

- **Ingest** purchase orders, delivery notes, warehouse barcode scans and supplier invoices.
- **Normalise** supplier item codes (for example `KN/7741-B`, `kn 7741 b`, `KN_7742_A`) to one internal SKU.
- **Reconcile** each order line along the chain Ordered → Delivered → Scanned → Invoiced, and categorise any discrepancy.
- **Investigate** each discrepancy with an AI-written claim summary, an investigation case and a human approval step.
- **Update inventory** automatically for lines that match on all four documents.

The system is four n8n workflows sharing one Google Sheets database. Gmail carries the approval, escalation and alert emails, and an AI model (first Google Gemini, now a Baseten-hosted model) writes the claim summaries.

2. Architecture

The four workflows share their data through one spreadsheet; the only direct call is Workflow 2 starting Workflow 3. Workflow 1 writes new documents, Workflow 2 reconciles them and hands discrepancies directly to Workflow 3, and Workflow 4 catches failures from the other three.



System architecture · 4 workflows, 1 database

Read it top to bottom: documents enter through Workflow 1, Workflow 2 splits each order line into matched or discrepancy, and every workflow records its results in the database on the right. The external services are Google Sheets (storage), Gmail (approval, escalation and alert emails) and the AI model behind Workflow 3's AI Agent.

3. Data store: one spreadsheet, eight tabs

All persistent data lives in one Google Sheets spreadsheet with eight tabs. Every document row carries a `status` column, which is how the system tracks where each record is in its lifecycle (section 8).

Tab	Holds	Key columns	Written by
Purchase_Orders	One row per order line	`line_id`, `po_number`, `supplier_id`, `internal_sku`, `ordered_qty`, `unit_price`, `status`, `created_at`	WF1 (append), WF2 and WF3 (status)

Tab	Holds	Key columns	Written by
Deliveries	Delivery note lines	`delivery_id`, `po_number`, `supplier_id`, `supplier_sku_raw`, `delivered_qty`, `delivery_date`, `status`	WF1, WF2
Warehouse_Scans	Barcode scan counts	`scan_id`, `po_number`, `scanned_sku_raw`, `good_qty`, `damaged_qty`, `scan_date`, `scanned_by`, `status`	WF1, WF2
Invoices	Supplier invoice lines	`invoice_id`, `po_number`, `supplier_id`, `supplier_sku_raw`, `invoiced_qty`, `unit_price`, `invoice_date`, `status`	WF1, WF2
SKU_Map	Supplier code → internal SKU lookup	`supplier_id`, `supplier_sku_clean`, `internal_sku`	Maintained by hand
Inventory	Stock on hand	`internal_sku`, `on_hand`, last-updated date	WF2 (matched lines only)
Investigations	One case per discrepancy	`case_id`, `po_number`, `internal_sku`, `categories`, the four quantities, `claim_amount`, `ai_summary`, `recommended_action`, `severity`, `approval_status`, `created_at`, `resolved_at`, `line_id`	WF3
Audit_Log	Every state change and error	`log_id`, `timestamp`, `po_number`,	WF2, WF3, WF4

Tab	Holds	Key columns	Written by
		`internal_sku`, `event`, `actor`, `old_value`, `new_value`, `details`	

Two rules matter for SKU_Map. Its headers must be exactly `supplier_id`, `supplier_sku_clean` and `internal_sku`, and `supplier_sku_clean` holds the code with every non-alphanumeric character removed, in capitals (`KN7741B`, not `KN/7741-B`). The current mappings are SUP-KANO KN7741B → INV-RICE-50KG, SUP-KANO KN7742A → INV-SUGAR-1KG, SUP-LAGOS LG5520 → INV-OIL-5L, SUP-LAGOS LG5531 → INV-SOAP-BAR and SUP-ENUGU EN8800X → INV-FLOUR-10KG.

4. Workflow 1 — Ingest documents

Workflow 1 receives documents over HTTP, checks them and appends them to the right tab with status `pending`. It is the system's front door.

1. **Webhook ingest** — `POST /webhook/warehouse-ingest`. A body is either one record or `{doc_type, rows:[…]}` for a batch.
2. **Validate and shape** (Code node) — rejects the whole request if `doc_type` is not PO, DELIVERY, SCAN or INVOICE, if a required field is missing, or if a quantity or price is not a non-negative number. It then upper-cases `po_number`, generates an ID where none is given (`L-`, `D-`, `SC-` or `INV-` + timestamp + row number), fills defaults (today's date, `damaged_qty` 0, `scanned_by` "unknown") and sets `status = pending`.
3. **Route by doc_type** (Switch) — sends each record down one of four branches.
4. **Append** to Purchase_Orders, Deliveries, Warehouse_Scans or Invoices.

Document	Required fields
PO	`po_number`, `supplier_id`, `internal_sku`, `ordered_qty`, `unit_price`
DELIVERY	`po_number`, `supplier_id`, `supplier_sku_raw`, `delivered_qty`
SCAN	`po_number`, `scanned_sku_raw`, `good_qty`
INVOICE	`po_number`, `supplier_id`, `supplier_sku_raw`, `invoiced_qty`, `unit_price`

Workflow 1 must be **published** for its production URL to exist; an unpublished workflow returns 404 "webhook is not registered". Known limitation: the webhook replies 200 OK as soon as a request arrives, before validation runs, so a sender is never told that a record was rejected.

5. Workflow 2 — Reconcile receiving

Workflow 2 takes every `pending` order line that has all four documents, matches them, updates stock for clean matches and hands every discrepancy to Workflow 3. It runs every 5 minutes on a schedule, or on demand from a manual trigger used for testing.

Step by step:

1. **Read** all six reference tabs: Purchase_Orders, Deliveries, Warehouse_Scans, Invoices, SKU_Map and Inventory. Each read node is set to execute once.
2. **Normalize and four-way match** (Code node) does the core work:
    - keeps only rows with status `pending`;
    - skips any order line that does not yet have a delivery, a scan *and* an invoice, so it stays `pending` until the missing document arrives;
    - cleans every supplier code (capitals, non-alphanumerics removed, so `KN/7741-B` becomes `KN7741B`) and looks it up in SKU_Map under the key `supplier_id|clean code`;
    - totals delivered, scanned-good, scanned-damaged and invoiced quantities for the line's internal SKU;
    - assigns categories and a claim amount (tables below);
    - for a clean match, adds the scanned good quantity to that SKU's stock.
3. **Matched or discrepancy** (Switch) splits the lines.
4. **Writes for matched** and **Writes for discrepancy** (Code nodes) turn each line into the status updates for its four document rows, an inventory update (matched only) and an Audit_Log row. Each item carries an internal `target` field naming its tab.
5. **Route write** (Switch) sends each item to its write node: Update Purchase_Orders / Deliveries / Warehouse_Scans / Invoices status, Update Inventory on_hand, Append Audit_Log.
6. **Call '3 - Investigate discrepancy'** (Execute Workflow) passes all discrepancy lines to Workflow 3 at once, with **Wait For Sub-Workflow Completion off**.

Discrepancy categories. A line with an unknown or wrong supplier code gets only the SKU category; otherwise every rule that applies is listed.

Category	Rule
`unmapped_sku`	A supplier code is not in SKU_Map
`sku_mismatch`	A code maps to an internal SKU that is not on this PO
`under_shipment`	delivered < ordered
`over_shipment`	delivered > ordered
`receiving_shortage`	good + damaged < delivered
`receiving_overage`	good + damaged > delivered
`damage`	damaged > 0
`over_billing`	invoiced > good

A line with no category is **matched**.

Claim amount. If invoiced > good, the claim is (invoiced − good) × unit price. Otherwise, if the line is under-shipped, it is (ordered − good) × unit price. SKU problems always give 0, because the quantities cannot be trusted.

The Call node must not wait for Workflow 3. Workflow 3 pauses for email approvals, so with waiting on, Workflow 2 stalls mid-run and its remaining writes never happen (section 9, step 22).

6. Workflow 3 — Investigate discrepancy

Workflow 3 turns each discrepancy into an AI-written investigation case, asks a person to approve or reject it by email, and records the decision. It is a sub-workflow: its only trigger is a call from Workflow 2.

1. **When called by workflow 2** — Execute Workflow Trigger, input mode "Accept all data". It receives every discrepancy line from Workflow 2.
2. **Loop over cases** — processes one case at a time. Because each case waits for an email reply, the approval emails arrive **one after another**, not all at once.
3. **Build prompt** (Code node) — stops with a clear error if a case has no `po_number` or `line_id`, so an empty input can never reach the AI. Otherwise it assembles the instructions and the case data.
4. **AI Agent** (with a Baseten chat model; previously a Gemini chat model, and before that a direct HTTP call to Gemini). Its **Prompt (User Message)**, in Expression mode, carries both the instructions and the case: the instruction text, then `Case data:` followed by `JSON.stringify($json.case)`, then a final line demanding the JSON object only. No structured output parser is attached. Retry On Fail is on.
5. **Parse AI response** (Code node) — accepts the answer whether it arrives as plain text, a parsed object or the raw Gemini format. It strips `<think>` blocks and ``` fences, extracts the JSON object, and enforces three guards:
    - the summary **must mention the case's own PO number** (this blocks invented cases);
    - `recommended_action` must be one of `file_supplier_claim`, `request_credit_note`, `request_replacement`, `fix_sku_mapping`, `accept_variance`, otherwise it falls back to `file_supplier_claim`;
    - `severity` must be low, medium or high, otherwise it falls back to medium.

    It also builds a unique `case_id` from the line ID and a timestamp.
6. **Create investigation case** — appends the case to Investigations with `approval_status = pending`.
7. **Human approval** — a Gmail send-and-wait email with Approve and Reject options. The run pauses here until the reviewer answers.
8. **Approved?** (IF) — **Build decision writes** prepares three updates: the Investigations row (`approval_status`, `resolved_at`), the PO line status (`approved` or `rejected`) and an Audit_Log row (`CLAIM_APPROVED` or `CLAIM_REJECTED`, actor `human:reviewer`).
9. **Escalate and notify** — on rejection only, an escalation email is sent.
10. **Merge writes** returns to the loop for the next case.

The AI is told to use only the figures provided. In testing this held: once the case data reached the model correctly, every summary quoted the real PO, line, quantities and supplier codes.

7. Workflow 4 — Error handler and retries

Failures are handled in two layers: nodes retry on their own first, and only a failure that survives the retries reaches the error workflow.

Layer 1: node retries. Each Google Sheets node and the AI Agent has Retry On Fail switched on (3 tries, a few seconds apart). The Gmail approval node in Workflow 3 has no retry, because a retry could send the reviewer a second approval email.

Layer 2: Workflow 4, "4 - Error handler".

1. **Error Trigger** — starts whenever a linked workflow fails.
2. **Shape error record** (Code node) — collects the workflow name, the failed node, the error message, the execution ID and a link to the failed run. It handles both execution errors and errors raised inside a trigger.
3. **Alert admin** (Gmail) — emails those details. If this email fails, the workflow continues to the log.
4. **Audit_Log columns only** + **Append Audit_Log** — writes a `WORKFLOW_ERROR` row, with the execution details in `details`.

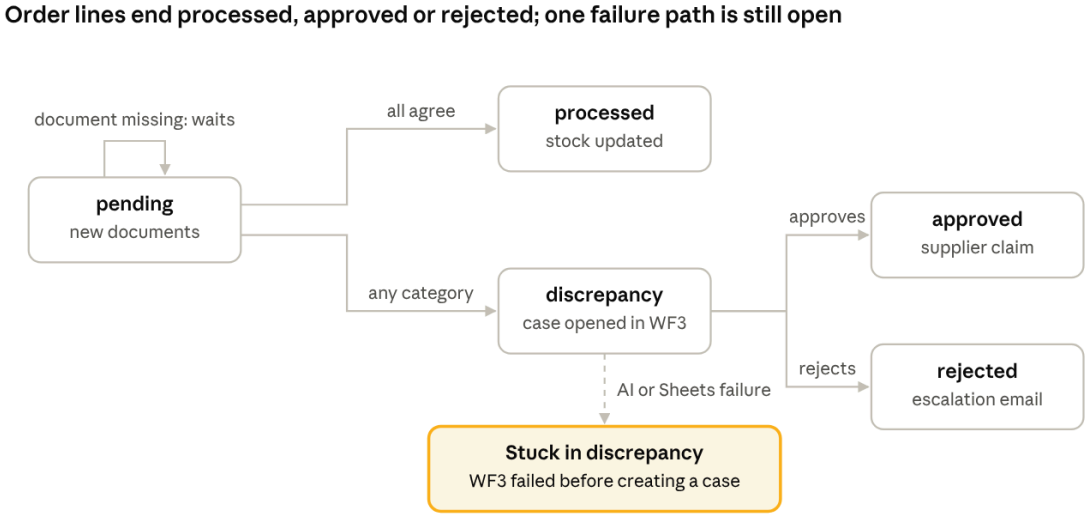
Setup: publish Workflow 4, then in the Settings of Workflows 1, 2 and 3 set **Error workflow** to "4 - Error handler".

Limitation: n8n calls the error workflow only for production runs (webhook, schedule, or a sub-workflow called from those). A run started with Execute workflow in the editor shows its error on the canvas only.

Test: post `{"doc_type":"X"}` to the production webhook. Validate and shape rejects it, and within seconds an alert email and a `WORKFLOW_ERROR` audit row should appear. Workflow 4 has been built and the setup steps given; its live test has not yet been confirmed.

8. State lifecycle

Every order line moves through at most three states, recorded in the `status` column of its PO, delivery, scan and invoice rows. The investigation case keeps its own `approval_status` (pending, then approved or rejected) in step with the PO line.



Order line states · 5 states, 1 open failure path

A line waits in `pending` until all four documents exist, as PO-NG-3108 did in testing. The amber path is the one dead end: if Workflow 3 fails before creating a case, the line stays in `discrepancy` and no later run picks it up. Closing it is part of the planned failure demonstration (section 12).

9. Build and test log, step by step

Workflows 1 to 3 already existed when this phase began. The 24 steps below, in the order they happened, took the system from separate workflows to a connected, tested chain. Most problems were configuration, not logic: the matching code was correct throughout.

#	What happened	Cause	Fix / outcome
1	Workflow 2's discrepancy branch ended at a placeholder node ("Step 4 goes here"); no error workflow existed	Build not finished	Placeholder replaced by an Execute Workflow call to Workflow 3; Workflow 4 (error handler) built
2	Offline test of Workflow 3's code nodes with 16 mocked AI replies	—	Found: one bad AI reply stops the whole batch and strands the PO in `discrepancy`; actions not validated; case IDs could collide. The last two were fixed in step 5; the first is still open (section 12)

#	What happened	Cause	Fix / outcome
3	First end-to-end script (`test_e2e.ps1`, PO-2001 to 2004) simulated through the Workflow 1 and 2 code	—	Expected results confirmed; found that sending the same data twice turns every PO into an over-shipment
4	Every webhook call returned 404	Workflow 1 not published	Workflow 1 published
5	After the Gemini HTTP node was replaced by an AI Agent: "Gemini returned no answer"; the reply was cut off and described an invented PO-1044	Parser expected the old format; token limit too low; case data not reaching the model	Parse AI response rewritten (reads agent output, PO-number guard, action check, unique case ID); token limit raised to 1024
6	Guard fired: summary "does not mention undefined", invented PO-4412	Workflow 3 run from the editor with no input, so it received an empty item	Guard added to Build prompt; test data pinned on the trigger
7	"c is not defined" in Build prompt	Guard lines pasted outside the loop	Whole Build prompt node replaced
8	Guard fired: summary about PO-9021 instead of PO-TEST-001	Prompt field held only the instruction text copied from the code, with no case data	Prompt switched to Expression mode with the case JSON
9	Model wrote a prose letter, no JSON	System message not reaching the model	Structured Output Parser tried
10	"fetch failed", then 503 "high demand" from `gemini-3-flash-preview`	Network drop, then an overloaded preview model	Retry On Fail on the AI Agent; stable model advised
11	Gemini quota ran out; with Baseten: "Model output doesn't fit required format"	Open model's output did not satisfy the strict parser	Parser removed; Parse AI response also strips `<think>` blocks
12	"Unexpected token '/'… is not valid JSON"	JavaScript pasted into a JSON field	Code moved into the Parse AI response Code node

#	What happened	Cause	Fix / outcome
13	Parser error persisted	Require Specific Output Format still on	Switched off; parser node deleted
14	Model wrote a markdown report	Instructions still not reaching the model	Instructions and case data both placed in the Prompt (User Message). **Workflow 3 passed**
15	Workflow 2 connected to Workflow 3 on the canvas	—	Publishing rule set: Workflows 1, 3 and 4 published; Workflow 2 saved but unpublished during tests
16	New test data PO-2101 to 2105 sent; the old script was also run by mistake	Two script files with similar names	Old file removed later; new data sets get distinct names
17	Append Audit_Log: "The resource you are requesting could not be found"	Node pointed at a wrong spreadsheet ID or tab name	Document and Sheet chosen with **From list**
18	Audit_Log showed every PO as `unmapped_sku`; the AI summaries correctly reported the mapping failure	SKU lookup returned nothing (cause found in step 21)	Evidence that the AI no longer invents figures
19	"Column names were updated after the node's setup" on Update Invoices status	The `target` column was deleted from the sheet; the node kept the old column list	Columns refreshed; write nodes set to Map Each Column Manually
20	Second data set built (`test_e2e_v2.ps1`, PO-NG-31xx, three suppliers, five SKUs, messier codes)	—	SKU_Map and Inventory rows added

#	What happened	Cause	Fix / outcome
21	All POs `unmapped_sku` again; Read SKU_Map returned `[{}]`	**Read SKU_Map was reading the Audit_Log tab**	Sheet set to SKU_Map. Matching then worked: RICE 15 → 55, OIL 0 → 120, two `MATCHED_INVENTORY_UPDATED` rows
22	Discrepancy invoices stayed `pending` and no `DISCREPANCY_FOUND` rows; the run ended "cancelled manually"	The Call node waited for Workflow 3, which was waiting for an email reply	Wait For Sub-Workflow Completion switched off
23	Re-run: discrepancies written correctly, but PO-NG-3101 and 3102 stayed `pending`; PO numbers were still 31xx	`$Run` edit not saved; matched branch not writing (suspected: `matched` output disconnected)	**Open** — check the `matched` → Writes for matched connection
24	Third data set built (`test_e2e_v3.ps1`) with PO numbers generated from the date and time	Avoids manual `$Run` edits	Ready to run once step 23 is resolved

10. Test data and how to run a test

Tests are driven by PowerShell scripts that post four batches (POs, deliveries, scans, invoices) to Workflow 1's webhook. Every data set was first run through the real Workflow 1 and 2 code offline, so each expected result below is computed, not guessed.

Script	PO numbers	Suppliers	Purpose
`test_e2e.ps1` (original)	PO-2001 to 2004	S01, S02, S03	First chain test; superseded
`test_e2e.ps1` (revised)	PO-`$Run`01 to 05	S01, S02, S03	Adds an unmapped code; superseded
`test_e2e_v2.ps1`	PO-NG-`$Run`01 to 08	SUP-KANO, SUP-LAGOS, SUP-ENUGU	Adds SKU mismatch and a missing invoice; `$Run` edited by hand

Script	PO numbers	Suppliers	Purpose
`test_e2e_v3.ps1` (current)	PO-ABJ-*ddHHmm*-01 to 08	SUP-KANO, SUP-LAGOS, SUP-ENUGU	New scenarios; PO numbers unique automatically

Expected results for the current data set (v3):

Line	Scenario	Category	Claim	Effect	Reviewer action
-01	Rice, all four documents agree	matched	—	INV-RICE-50KG +25	none
-02	Sugar, all four agree	matched	—	INV-SUGAR-1KG +150	none
-03	Oil: 200 ordered, 170 delivered and invoiced	under_shipment	285,000	case	Approve
-04	Soap: 10 damaged, only 290 good invoiced	damage	0	case	Approve
-05	Flour: 48 delivered against 40 ordered	over_shipment	0	case	Approve
-06	Oil: 100 billed, 93 counted	receiving_shortage, over_billing	66,500	case + escalation	**Reject**
-07	Oil code delivered against a soap PO	sku_mismatch	0	case	Approve
-08	No scan yet	(not reconciled)	—	stays `pending`	none

Running a test:

1. Check the setup: Workflow 1 and 3 (and 4) published; Workflow 2 saved, not published; every Sheets node set with From list to the right tab; Call node not waiting; no pinned data on Workflow 3's trigger.
2. Note the current Inventory values.
3. Run `.\test_e2e_v3.ps1`. The last line must begin "Done v3".
4. Check the four document tabs: each PO appears once, status `pending`.
5. Run Workflow 2 with the Manual trigger. It should finish as Succeeded within seconds, with 2 items into Writes for matched and 5 into the Call node.
6. Answer the approval emails as they arrive, one at a time.
7. Check Purchase_Orders, Inventory, Investigations and Audit_Log against the table above.

Two cautions. Never send the same PO numbers twice, because the duplicates are summed and every line becomes an over-shipment. And stop any Workflow 3 run left "Waiting" from an earlier attempt before starting a new one.

11. Compliance with the 12 mandatory components

Ten of the twelve components are met (two with caveats, #2 and #9), one is partial (#10) and one is not yet built (#11). The Group 11 domain requirements (ingestion, SKU normalisation, the four-quantity chain, discrepancy categories, AI claim summaries with cases, and inventory updates for matches) are all met.

#	Component	How it is met	Status
1	2 triggers	Webhook (WF1) and Schedule (WF2); also Manual, sub-workflow and Error triggers	Met
2	3+ external integrations	Google Sheets, Gmail, AI model API	Met, but weak: Sheets and Gmail are both Google, and the AI may be counted under #4
3	Persistent storage	Google Sheets, eight tabs	Met
4	AI component	AI Agent writing claim summary, action and severity	Met
5	Code node	Validation, matching, prompt building, AI parsing, error shaping	Met
6	Sub-workflow	WF2 calls WF3; WF4 is the error sub-workflow	Met

#	Component	How it is met	Status
7	Loop / batch	Loop over cases (WF3); batch rows (WF1)	Met
8	Human approval / escalation	Gmail send-and-wait approval; escalation email on rejection	Met
9	Retry / error handling	Node retries + WF4	Met once WF4's live test is confirmed
10	Audit log	Audit_Log with event, timestamp, actor, before/after values, payload	**Partial**: no execution ID on normal rows, no payload signature
11	Live failure scenario	Not yet scripted	**Gap**
12	State management	`status` on every document row and `approval_status` on every case	Met, with one dead end (section 8)